

Hello World

```
• fn main() {
•     println!("Hello, World!");
• }
```

Will this still work as a Hello World app in a year, we don't know — it most probably will — but it sure does right now! Rust keeps things simple when you're trying to do simple things while still having enough power to accomplish complex things when needed.

Sort

```
• fn insertion_sort(uarr: &mut [int]) {
•     for idx in range(1u, uarr.len()) {
•         let curr_value = uarr[idx];
•         let mut pos = idx;
•         while pos > 0 && uarr[pos - 1] > curr_value {
•             uarr[pos] = uarr[pos - 1];
•             pos -= 1;
•         }
•         uarr[pos] = curr_value;
•     }
• }
```

On a glance this might look like just some C or C++ code, but there are some interesting differences. For one, there are no brackets around the expressions following `for` and `while`. Another noticeable difference is that you do not need to declare the type of a variable if that is obvious from the declaration.

You will also notice that we declare `'curr_value'` as `'let curr_value = uarr[idx];'` while for `'pos'` we declare `'let mut pos = idx;'`. This is because by default variables in Rust cannot be changed once set; so if we want a variable that need to be constantly modified, we need to explicitly declare it as a mutable variable. Since we want to modify the incoming array to sort it, the array needs to be mutable as well.

Tools and Learning Resources

Given that Rust is still heavily in development, and the syntax of today might not work tomorrow, the best place to learn the language is the official website. While other tutorials and blog posts that introduce the